IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2023.1120000

Transformer and Meta-Reinforcement Learning-Based Task Offloading for MEC systems in Dynamic Environments

YINTAO LI[1], CHUANGUO TANG[1], BOHAO HOU[2], FEIBO JIANG[1], YUHANG FU[3]

[1]School of Information Science and Engineering, Hunan Normal University, Changsha 410205, China (e-mail: liyintao320@163.com, jiangfb@hunnu.edu.cn)
[2]School of Computer Science, Hunan University Of Technology and Business, Changsha 410205, China (e-mail: hbh2632937318@gmail.com, dlj2017@hunnu.edu.cn)
[3]Hunan Economic Institute Electric Power Design Co.,Ltd, Changsha 410081, China(e-mail: fuyh1@hn.sgcc.com.cn)

Corresponding author: Chuanguo Tang (e-mail: 202220294014@hunnu.edu.cn).

ABSTRACT In mobile edge computing systems, efficient decision-making for task offloading is crucial for enhancing system performance. Currently, existing reinforcement learning-based offloading methods face several primary challenges, including local feature perception, low sample efficiency, long training times, and poor stability. To address these issues, this paper proposes efficient task offloading methods based on transformer and meta-reinforcement learning. Firstly, a novel transformer architecture, Gated Transformer-XL is introduced, allowing the agent to adaptively capture various features and relationships in task offloading decisions and transform them into representations suitable for meta-reinforcement learning algorithms. Secondly, a meta-reinforcement learning framework, Gated Transformer-XL based Proximal policy optimization and Reptile (GTrXL-PR), is proposed, which offers high sample efficiency for new tasks. Even with limited computational resources, user devices can quickly train using local data combined with meta-policies. Finally, Reptile is combined with the proximal policy optimization algorithm to reduce training time by ignoring second-order derivatives, thereby enhancing the stability of the training process. The feasibility and effectiveness of the proposed algorithm are validated through experimental simulations. In experimental simulations, the proposed method achieved approximately a 1.96% reduction in task completion latency across varying rates and numbers of subtasks compared to conventional reinforcement learning methods. The initial latency was also reduced by approximately 0.12% relative to traditional algorithms, validating the feasibility and effectiveness of the proposed algorithm. For more details, the core code is available at https://github.com/liyintao23/Gtrxl_PR.

INDEX TERMS Mobile edge computing; Task offloading; Meta-learning; Reinforcement learning.

I. INTRODUCTION

Mobile Edge Computing (MEC) is an emerging computing paradigm that aims to meet the growing computational demands and low-latency service requirements by deploying computing resources at the network edge. With the rapid increase in the number of Internet of Things (IoT) devices, smartphones, and other mobile devices, the traditional cloud computing model has become inadequate to effectively handle the massive data processing and real-time service demands. MEC brings computational capabilities closer to the data source, significantly reducing transmission latency, improving data processing efficiency, and supporting a richer variety of applications [1]–[3].

In MEC systems, task offloading decisions are of critical importance. Task offloading involves the rational distribution of computational tasks between local devices and edge servers to optimize system performance. Effective task offloading can reduce computational latency, save energy, and improve overall network resource utilization [4], [5]. However, the task offloading problem is often formulated as a Mixed-Integer Nonlinear Programming (MINLP) problem, and the dynamic and complex environment presents numerous challenges for task offloading decisions in MEC systems [6]–[8]. Factors such as channel state information, availability of computing resources, and the mobility of user devices can all impact task offloading decisions [9]–[11].

Task offloading decisions in MEC systems often face highly dynamic and uncertain environments. Traditional op-

timization methods struggle to adapt to these changes in real-time and perform poorly in complex strategy optimization problems. In contrast, Reinforcement Learning (RL) methods can adaptively learn efficient task-offloading strategies through continuous interaction with the environment, enabling real-time optimization in dynamic settings.

To address these challenges, researchers have proposed various RL-based task-offloading methods in recent years, aiming to optimize task-offloading decisions through intelligent means. However, these methods still have limitations as follows:

- Traditional RL methods suffer from low sample efficiency and inadequate model adaptability, making it difficult to achieve optimal performance in complex and dynamic environments [12], [13]. Many scholars have turned their attention to meta-learning. Meta-learning, or learning to learn, enables artificial intelligence to autonomously learn new tasks based on past experiences. With the accumulation of experience from previous tasks, meta-learning exhibits superior performance when encountering new tasks, allowing for faster learning with fewer samples [14].
- Despite the remarkable performance of meta-learning in enhancing model adaptability and sample efficiency, it still faces several challenges when processing trajectory data. Trajectory data often contains long sequences of states and actions, and traditional neural networks tend to encounter issues like vanishing or exploding gradients when dealing with long-term dependencies. This problem hinders the model's ability to effectively capture long-term relationships.
- Traditional RL methods, such as policy gradient methods, can exhibit significant instability during the training process. This instability arises from multiple factors, including the sensitivity of these methods to hyperparameter settings, the inherent variability in stochastic gradient updates, and the potential for overly aggressive policy updates. As a result, these methods often lead to fluctuations in model performance, making it challenging to achieve consistent and reliable results. This instability not only hampers learning efficiency but also poses difficulties in practical applications where stable performance is crucial.

Meta-reinforcement learning shows great potential in solving task offloading problems in MEC systems under dynamic environments. Compared to learning from scratch, meta-reinforcement learning can acquire algorithms more efficiently. These algorithms can accumulate experience from past tasks and quickly adapt to and handle new tasks even when the environment changes [15]. This paper aims to develop a meta-reinforcement learning task offloading decision framework based on Gated Transformer-XL, which combines the powerful feature-capturing capabilities of Transformers with the efficient algorithms of meta-reinforcement learning. Through this innovative approach, we seek to enhance the adaptability and decision-making efficiency of MEC systems in complex dynamic environments, thereby promoting the practical application of MEC technology. The ultimate goal of this research is to achieve more intelligent task offloading strategies, improve user experience, reduce latency, and lay the groundwork for future more complex computing scenarios.

This paper proposes an innovative task offloading solution based on transformer and meta-reinforcement learning to address the issues of local feature perception, low sample efficiency, and long training times with poor stability in the MEC systems. The key contributions of this paper are as follows:

- We introduce a novel transformer architecture to enhance the global feature capture capability. We incorporate the GTrXL architecture for task offloading decisions in MEC systems, enabling the agent to adaptively capture various complex features and relationships inherent in these decisions. The gated mechanism in GTrXL allows the model to effectively handle long sequence dependencies and nonlinear relationships, thereby improving the accuracy and effectiveness of the decision-making process.
- We propose a meta-reinforcement learning-based task offloading framework which is called GTrXL-based Proximal Policy Optimization (PPO) and Reptile (GTrXL-PR). This framework combines the Transformer-XL and meta-reinforcement learning algorithms, enhancing the model's adaptability. It enables rapid adaptation to new tasks, facilitating efficient task-offloading decisions even with limited computational resources.
- To reduce training time and improve training stability, we combine the Reptile algorithm with the PPO algorithm in meta-reinforcement learning, ignoring second-order derivatives. This approach accelerates the training speed and ensures stability throughout the training process. This combination not only enhances the performance of task-offloading decisions but also demonstrates high reliability in practical applications.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III describes the system model and problem formulation. Section IV outlines the detailed algorithm design of the GTrXL-PR. Section V presents simulation results, followed by the conclusion in Section VI.

II. RELATED WORK

Existing studies of intelligent task offloading methods and transformer-based RL mainly focus on the following aspects:

A. HEURISTIC SEARCH-BASED TASK OFFLOADING

Some studies model the task offloading problem as an optimization problem and solve it using mathematical optimization tools. For example, methods such as convex optimization, game theory, and Lagrangian relaxation are used to minimize

computation delay and energy consumption. These methods theoretically guarantee good performance, but in practical applications, the solving process is complex and computationally intensive, making it difficult to respond to changes in the MEC environment in real-time [16]. Early task offloading decision methods were mainly based on heuristic algorithms, such as greedy algorithms, genetic algorithms, and particle swarm optimization. These methods solve specific task-offloading problems by designing algorithm rules, offering simplicity and low computational complexity. Recent advancements have incorporated hybrid methods like Switched Auto-Regressive Neural Control (S-ANC) and logic predictive controllers, which combine heuristic strategies with predictive mechanisms to improve decision-making and adaptability in complex environments [17], [18]. However, since heuristic algorithms typically require manual design for specific problems, they struggle to adapt to the dynamic and diverse MEC environment and complex task characteristics [19], [20].

B. REINFORCEMENT LEARNING-BASED TASK OFFLOADING

With the development of machine learning technologies, researchers have begun exploring the use of machine learning methods for task offloading decisions. For example, supervised learning methods train models based on historical data to predict optimal offloading strategies. However, supervised learning methods generally rely on large amounts of labeled data and are slow to adapt to new tasks in dynamic environments [21]–[24]. In recent years, RL methods have gained widespread attention in task offloading decisions. RL methods continually optimize decision strategies through interaction with the environment, making them suitable for dynamic and uncertain environments. For example, algorithms like Deep Q-Network (DQN) and PPO have been applied to task offloading decisions, achieving good results [25], [26]. However, RL methods still face challenges in sample efficiency and training stability, especially in high-dimensional and complex task scenarios, where training time is long and results are unstable.

C. META REINFORCEMENT LEARNING-BASED TASK OFFLOADING

As an emerging learning paradigm, meta-learning has gradually been applied to task offloading decisions in recent years. Meta-learning extracts common knowledge through multi-task learning to improve the model's adaptability to new tasks. Qu et al. [27] proposed a Deep Meta-Reinforcement Learning Offloading (DMRO) algorithm, which combines multiple parallel Deep Neural Networks (DNNs) with Q-learning to make fine-grained offloading decisions. This approach enables the rapid and flexible acquisition of optimal offloading strategies in dynamic environments. Chen et al. [28] proposed a meta-reinforcement learning-based cache-assisted computation offloading method (MCCOM). This method enables rapid offloading decisions with minimal gradient updates and samples, thereby efficiently achieving offloading and resource allocation decisions. Dai et al. [29] proposed a Seq2Seq-based Meta Reinforcement Learning algorithm for multi-task offloading (SMRL-MTO). Specifically, a bidirectional gated recurrent unit integrated with an attention mechanism was designed to encode sequential offloading operations and determine offloading actions by displaying different preferences for different parts of the input sequence. Hossain et al. [30] proposed a computation task offloading and power control mechanism based on Meta-Reinforcement Learning (MRL) for User Equipment (UE) in resource-constrained MEC networks. This mechanism addresses the NP-hard problem of offloading strategy optimization. Yang et al. [31] proposed a novel energy-efficient algorithm for multi-user adaptive edge computing offloading in vehicular networks, aiming to optimize the allocation of computing resources. The core of this strategy is a framework based on meta-reinforcement learning, which excels in addressing the dynamic and complex nature of Internet of Vehicles (IoV) environments.

D. TRANSFORMER BASED TASK OFFLOADING

With the remarkable success of transformer models in supervised learning, many scholars have attempted to apply transformers to RL to address sample efficiency and long-dependency issues. GTrXL [32] is the first effective approach to using the transformer as the memory architecture to handle reinforcement learning trajectories. GTrXL modifies the Transformer-XL architecture [33] by using identity mapping reordering to provide a skip path from the temporal input to the transformer output, stabilizing the training process from the beginning. Zhao et al. [34] proposed a multi-agent deep reinforcement learning approach that integrates Transformer models to optimize task offloading and resource management in UAV-assisted MEC. Yang et al. [35] proposed LTransformer, a transformer-based framework aimed at optimizing task offloading in Vehicular Edge Computing (VEC). The core of this framework includes pre-training modules, input modules, encoding-decoding modules, and output modules, significantly enhancing the accuracy and real-time performance of task offloading. Additionally, Gholipour et al. [36] proposed a DRL method called TPTO, which leverages transformer networks and PPO to offload tasks related to IoT applications in edge computing environments. Wu et al. [37] explored the use of Seq2Seq models for mobility prediction to optimize task offloading and resource allocation in edge computing. By integrating deep reinforcement learning with transformers, they significantly improved resource utilization and task processing efficiency.

In summary, existing task-offloading decision methods have their own limitations, making it difficult to simultaneously achieve optimal performance in terms of sample efficiency, adaptability, training time, and stability. This paper proposes a GTrXL-PR framework based on meta-reinforcement learning, combining the powerful feature capture capabilities of Transformer-XL with the efficient train-

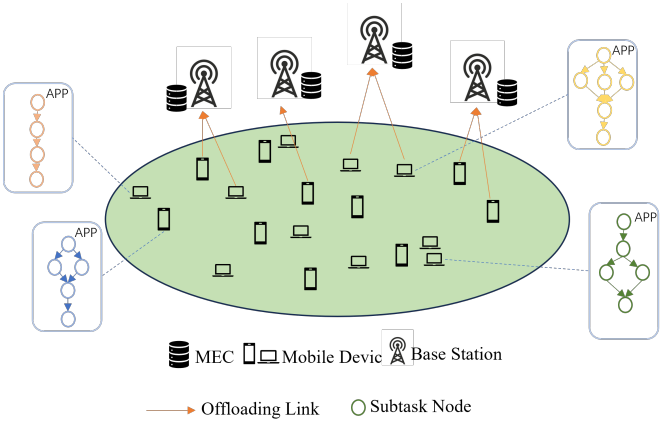

FIGURE 1. The System Model for MEC systems.

ing mechanisms of PPO and Reptile algorithms, aiming to address the above issues and provide an efficient and reliable task offloading decision solution.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. SYSTEM MODEL

As shown in Fig. 1, we consider a fine-grained task offloading scenario involving multiple UE and multiple MEC servers. Each UE has $M = \{1, 2, 3, \ldots, I\}$ applications. Each application consists of $I = \{1, 2, 3, \ldots, H\}$ temporally dependent subtasks. Therefore, the tasks discussed in this chapter are composed of subtasks from various device applications. The environment includes multiple heterogeneous tasks generated by different user devices. Each task can only connect to one channel at a time, and multiple tasks in the same channel may interfere with each other. Each task can only choose between computational offloading or local execution, and considering the limitations of the UE, it can only establish a connection with one MEC server at a time. Hence, even if many MEC servers are available, a UE can only choose one MEC server for offloading while executing an application, and consequently, subtasks can only be offloaded to one MEC server. Since applications can be decomposed into multiple correlated subtasks, this chapter represents an application as a Directed Acyclic Graph (DAG) $G = \{V, E\}$, where V is the set of vertices representing all subtasks, $|V|$ denotes the number of vertices in the DAG, i.e., the number of subtasks, and E is the set of edges representing the dependencies between subtasks. If adjacent nodes have a parent-child relationship, the child node can only operate once all parent tasks are completed. Each node can be described as a triplet $T_{i,j} = \{S_{i,j}, C_{i,j}, O_{i,j}\}$, where $S_{i,j}$ denotes the size of the received data of the task of node j of device i, $C_{i,j}$ denotes the number of CPU cycles required for the calculation of the task of node j of device i, and $O_{i,j}$ represents the size of the data output after the calculation of the task of node j of device i. In this system, each user device performs the inner loop training part of meta-learning, while the MEC servers execute the outer loop training part of meta-learning. The changes in the outer loop network parameters are synchronized across different MEC servers. The objective is to minimize task response latency by optimizing the offloading decisions for a series of tasks represented by multiple DAGs. This is achieved by finding an optimal set of initialization parameters, so that when the environment changes, especially when the number of DAG tasks changes and the differences between DAGs are significant, the network model can converge with fewer update steps using the outer loop training parameters of the MEC servers as initialization parameters, and the time to infer the offloading decision strategy is shorter.

For tasks choosing to offload computation to the MEC, the UE offloads the task to the edge server, which then executes the computational task and returns the result to the UE. In this context, the subtask j of device i is defined as $T_{i,j}$. Therefore, the task offloading process includes three sequential phases: the offloading phase, the edge execution phase, and the download computation output phase. According to Shannon's theorem, the uplink and downlink rates of the task transmission from device i to edge server k can be calculated by the following formula:

$$R_{i,k}^{ul} = R_{i,k}^{dl} = B \log_2 \left(1 + \frac{p}{N_0 + N}\right), \tag{1}$$

where B is the channel bandwidth, p is the transmission power when offloading the task to MEC, N_0 represents the environmental noise floor, and N denotes the impact on the channel caused by other devices simultaneously offloading tasks to the server chosen by device i.

The main goal of task offloading in this chapter is to seek the optimal strategy. Each subtask can be offloaded to MEC or local computing.

When task $T_{i,j}$ is offloaded locally, all preceding tasks of $T_{i,j}$ must have been completed. The local calculation time can be determined by the required CPU cycles and the computational capability of the user equipment f_i^{ue}, and can be expressed by the following formula:

$$t_{i,j}^{ue} = \frac{C_{i,j}}{f_i^{ue}}. \tag{2}$$

When task $T_{i,j}$ is offloaded to the MEC, the task execution process involves three steps. First, the device sends the task to the MEC through a wireless channel, resulting in an upload delay. Second, upon receiving the task, the MEC queues the task and then processes it, resulting in queuing delay and computation delay. Therefore, the delay when task $T_{i,j}$ is offloaded to the MEC consists of the upload delay, queuing delay, and computation delay. This can be expressed by the following formula:

$$t_{i,j}^{k} = \frac{S_{i,j}}{R_{i,k}^{ul}} + t_{i,j}^{k,\text{queue}} + \frac{C_{i,j}}{f^{k}}, \tag{3}$$

where the first item represents the upload time of task $T_{i,j}$ from task node j of device i to server k, the second item represents the queue time waiting for MEC resources and dependent node data to be ready, the third item represents the

MEC computing time, and f^k represents the MEC computing power. This paper uses the variable a_i to represent the node task offloading strategy of device i.

$$a_i = \begin{cases} 0, & \text{All subtasks are executed locally} \\ k, & \text{Offload subtasks to the MEC server } k \end{cases}, \quad (4)$$

where a_i is mainly used to evaluate the impact of occupied channels on other devices. If local execution is chosen, the channel is not occupied, and there is no resource impact on other tasks. When multiple tasks choose to execute on the same MEC, considering a first-come, first-served scheduling policy, each task arriving at the MEC must wait for the tasks that arrived before it to be completed before it can enter the execution state. The waiting delay $t_{i,j}^{k,\text{queue}}$ can be expressed as follows:

$$t_{i,j}^{k,\text{queue}} = \sum_{q_{j'} \in P_{\text{pre}}(q_j)} \left(BT(q_{j'}, k) + \frac{C_{i,q_{j'}}}{f^k} \right), \quad (5)$$

where $P_{\text{pre}}(q_j)$ represents the queue of pending tasks in the processing queue when task j arrives at MEC, $q_{j'}$ denotes a pending task node, $BT(q_{j'}, k)$ indicates the start time of each task in the pending queue, and $\frac{C_{i,q_{j'}}}{f^k}$ represents the execution time of each task in the pending queue. Since subtasks in a DAG are considered as offloading objects, the start execution time of each task is related to the completion status of all its predecessors. A task will start computing only after all its predecessor nodes have been completed and their results have been sent to the current node's location. For subtask node j, when selecting server k for edge offloading, its start processing time $BT(j,k)$ and finish processing time $FT(j,k)$ can be expressed as follows:

$$BT(j,k) = \max \left\{ \text{avail}\{0 \cup [k]\}, \max_{j' \in \text{pre}(j)} \left(FT(j') + C_{jj'} \right) \right\}, \quad (6)$$

$$FT(j,k) = \min \left\{ e_{i,j}^{k'} + BT(j',k') \mid k' \in \{0\} \cup \{k\} \right\}, \quad (7)$$

where $\text{avail}\{0 \cup [k]\}$ represents the available time of the server or local device. $\text{pre}(j)$ denotes the predecessor nodes of j. $C_{jj'}$ represents the data communication time between the two nodes, which depends on their execution locations and the size of the data. Therefore, $\max_{j' \in \text{pre}(j)} \left(FT(j') + C_{jj'} \right)$ indicates that all the necessary data for j is ready. Thus, taking the maximum value in Eq. (6) satisfies the two necessary conditions for task execution: data readiness and resource availability. $e_{i,j}^{k'}$ denotes the execution time of the subtask node, whether executed locally or offloaded. The finish time is the minimum of the local processing finish time and the offloading processing finish time.

B. PROBLEM FORMULATION

Tasks without subtasks are called end tasks, which are defined as the end task set $F = 1, 2, \ldots, n$. Therefore, only when all tasks in the end task set are completed is the completion time of the entire application, that is, the runtime delay, which can be expressed by the formula:

$$T^{\text{total}}(G) = \max_{\forall m \in F} \{ \max\{FT(m,k)\} \}, \quad (8)$$

This chapter considers delay-sensitive application scenarios. In order to optimize the user experience, it is necessary to find the optimal scheduling and unloading solution in multi-user scenarios by minimizing $T^{\text{total}}(G)$ for all users. Therefore, the objective function can be expressed as:

$$\begin{aligned} \text{minimize} \quad & \sum_{i=1}^{N} T^{\text{total}}(G) \\ \text{s.t.} \quad & \text{C1: } a_{i,j} \in \{0, \ldots, k\}, \\ & \text{C2: } a_{i,j} = k b_{i,j}, \quad b_{i,j} \in \{0,1\}, \\ & \text{C3: } \sum_{j=1}^{|V|} x_j f_j < f^k. \end{aligned} \quad (9)$$

where constraint C1 indicates the value range of the binary task offloading variable, constraint C2 indicates that any task can only choose $a_{i,j} = 0$ local calculation or $a_{i,j} = k$ to offload to MEC, and C3 indicates that the CPU frequency f_j allocated to each task offloaded to MEC cannot exceed the MEC maximum value f^k.

TABLE 1. Notations Used Throughout the Article

Notation	Description
$\|V\|$	The number of subtasks
$T_{i,j}$	The j-th subtask of the i-th device.
$S_{i,j}$	The size of received data
$C_{i,j}$	The required number of the CPU cycles
$O_{i,j}$	The output data size
$R_{i,k}^{ul}, R_{i,k}^{dl}$	The uplink and downlink rates of device i transmitted to edge server k
B	Channel bandwidth
p	The transmission power
N_0	Ambient noise
N	Channel impact
$t_{i,j}^{ue}$	local computation time
f_i^{ue}	computational capabilities of user devices
f^k	The computational capability of the k-th MEC
a_i	offloading strategy
$t_{i,j}^{k,queue}$	The waiting latency of the j-th task of the i-th device in the k-th MEC
$q_j^{'}$	Task node to be processed
BT	Task start time
FT	Task finish time
$avail\{0 \cup [k]\}$	The available time of the server or local device
$pre(j)$	the predecessor nodes of j
$C_{jj'}$	The data communication time between the two nodes
$e_{i,j}^{k'}$	The execution time of the subtask node
x_j	Whether the task is assigned to the j-th MEC
f_j	The CPU frequency allocated by the MEC to each task
F	The set of finish task
T^total	Total task latency
$RMHA$	Relative multi-head attention
E	The embedding vector
g	The gating layer function

IV. GTRXL META-REINFORCEMENT LEARNING ALGORITHMS

In deep reinforcement learning, the primary objective of meta-learning algorithms is to enable an agent to acquire an optimal strategy with only a small amount of experience when faced with new tasks. For instance, if an agent learns how to run in one environment, it can quickly learn how to jump in the same environment. In this paper, the agent can derive a strategy in one environment and, when the environment changes, it can adapt the original strategy to the new environment based on prior experience. From the previous sections, it is evident that GTrXL offers parallel computing and fast inference speed, while the PPO algorithm is easy to tune, stable in training, and performs well [38]. Therefore, the PPO algorithm and GTrXL network framework are chosen to solve the offloading decision problem. However, both PPO and GTrXL have low sample efficiency and require the collection of many samples for the network to converge. To address this issue, this chapter combines the meta-learning algorithm Reptile with the PPO algorithm, proposing the GTrXL-PR algorithm to handle the state sequences and policy information obtained from agent-environment interactions, followed by policy optimization and training using PPO. Reptile is an optimization-based first-order gradient meta-learning algorithm used to learn initial policy parameters across multiple tasks. When training on tasks, the meta-network parameters are updated by multiplying the parameter ϵ with the difference between each task network and the meta-network parameters. The meta-network parameters are updated multiple times based on the first-order gradients of each task network. In practical implementation, for each task, the agent interacts with the environment over multiple rounds to obtain state sequences. These state sequences are input into the GTrXL model for processing, generating a state representation that represents the task. In the Reptile algorithm, the state representation of each task is used to update the agent's policy parameters. Specifically, the parameters learned by the agent vary depending on the task, enabling the agent to quickly adapt to new tasks after learning a sufficient number of tasks. Throughout this process, the GTrXL model functions to transform state sequences into state representations, capturing long-term dependencies and trends in the sequences, and generating state representations that express the characteristics of the tasks, thereby supporting the learning process of the meta-reinforcement learning algorithm.

A. GTRXL MODEL

To reduce latency in the task offloading decision process, this chapter transforms the task offloading decision process into a model inference process. To capture the global relationships of past experiences and the importance of each experience, and to accelerate inference speed, we select the Transformer network architecture, which supports parallel computation, as the foundational network architecture. Additionally, considering the issue of stochastic policies that may arise when applied to reinforcement learning algorithms [39], we choose GTrXL, known for its more stable training in reinforcement learning, as the base architecture. The network structures of these two transformers are shown in Fig. 2.

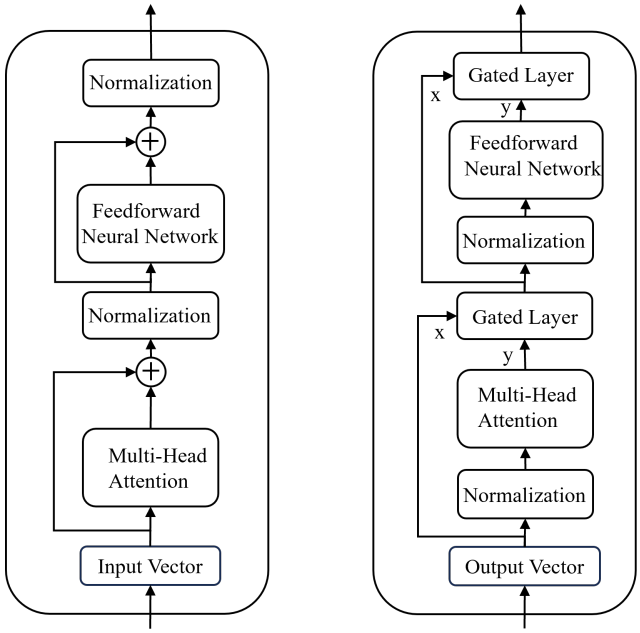

FIGURE 2. Transformer and GTrXL.

When applied to supervised learning, the transformer architecture presents high training difficulty. Typically, more complex learning rate adjustment schemes or targeted weight initialization methods are used to improve model performance and reduce training difficulty. However, these methods are not effective for reinforcement learning tasks. To address this issue, GTrXL first adjusts the position of the normalization layer to the input stream of the submodule and uses identity-mapped, re-ordered encoding blocks. A key advantage of this reordering is that it enables identity mapping from the input of the first layer to the output of the last layer of the transformer. This is in contrast to the standard transformer, which uses a series of layer normalization operations to nonlinearly transform state encodings. Therefore, assuming that the expected value produced by the submodule is close to zero at initialization, the state encoding is passed to the policy and value functions without transformation, allowing the agent to learn a Markov policy at the start of training. Secondly, a gating layer is used to replace the residual layer. The final GTrXL process is as follows:

$$\bar{Y}^{(l)} = \mathrm{RMHA}(\mathrm{LayerNorm}([\mathrm{StopGrad}(M^{(l-1)}), E^{(l-1)}])) \tag{10}$$

$$\bar{Y}^{(l)} = g^{(l)\mathrm{MHA}}(E^{(l-1)}, \mathrm{ReLU}(\bar{Y}^{(l)})) \tag{11}$$

$$\bar{E}^{(l)} = f^{(l)}(\mathrm{LayerNorm}(Y^{(l)})) \tag{12}$$

$$\bar{E}^{(l)} = g^{(l)}_{\mathrm{MHA}}(Y^{(l)}, \mathrm{ReLU}(\bar{E}^{(l)})) \tag{13}$$

where RMHA represents relative multi-head attention, E represents the embedding vector, the StopGrad function can

prevent the gradient from flowing backward during back-propagation, and g represents the gating layer function. The function can be expressed as:

$$g^{(l)}(x,y) = (1-z) \odot x + z \odot \hat{h} \tag{14}$$

$$r = \sigma(W_r^{(l)} y + U_r^{(l)} x) \tag{15}$$

$$z = \sigma(W_z^{(l)} y + U_z^{(l)} x - b_g^{(l)}) \tag{16}$$

$$\hat{h} = \tanh(W_g^{(l)} y + U_g^{(l)}(r \odot x)) \tag{17}$$

where r represents the reset gate, z represents the update gate, and $\hat{h}$ represents the hidden state. $W_g^{(l)}$ and $U_g^{(l)}$ represent the weight matrices, and $b_g^{(l)}$ represents the bias term. The pseudo-code of the encoder-decoder for trajectory encoding is shown in **Algorithm 1**.

Algorithm 1 GTrXL trajectory encoding

Initialize GTrXL parameters θ_{gtrxl}.
for each trajectory τ **do**
  Initialize hidden state $\hat{h}_0$.
  for $t = 1, 2, ..., T$ **do**
    Input the current state s_t and the previous hidden state $\hat{h}_{t-1}$ into GTrXL:
    $[h_t, \text{output}_t] \leftarrow \text{GTrXL}(s_t, \hat{h}_{t-1}; \theta_{\text{gtrxl}})$.
    Store the output $output_t$ as the encoded representation at time step t.
  end for
end for
Aggregate the encoded representations across the entire trajectory.
$trajectory \leftarrow \text{Aggregate}(outputs)$.
Update the policy θ_i for the specific task using the trajectory and the PPO algorithm with learning rate α:
$\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} J^{\text{PPO}}_{\text{trajectory}}(\theta_i)$.

B. GTRXL-PR ALGORITHM

The GTrXL-PR algorithm can be divided into two phases: the meta-training phase and the meta-testing phase, as shown in Fig. 3. The following sections provide a detailed introduction to these two phases.

1) Meta-Training Phase

To train a model that can adapt well to different new tasks, the algorithm selects N DAGs with the same number of tasks but varying widths, densities, and communication/computation ratios during the training phase. These N task samples are used as training data to update the parameters of the meta-network in a way that minimizes the expected loss across multiple task networks. The formula can be expressed as follows:

$$\min_{\theta} \mathbb{E}_{\tau \sim p(\tau)}[L_\tau(U_\tau^k(\theta))], \tag{18}$$

where $p(\tau)$ denotes the distribution of task trajectories, $U_\tau^k(\theta)$ represents the model parameters θ updated k, times after collecting a set amount of trajectory data, and $L_\tau(U_\tau^k(\theta))$ denotes the loss function.

Based on the above formula and the basic process of meta-learning, meta-training can be divided into two "loops": the "inner loop" and the "outer loop". In the inner loop, a task dataset $D_u = \{G^t\}_{t=1}^T$ is selected, where $t = 1, 2, ..., T$ represents the DAG task indices. Subsequently, a small number of tasks from D_u are randomly selected to form trajectory *tau* for model training and parameter updates, gradually reducing the loss for each task. During this process, the agent interacts with the environment using the policy generated by the Actor network to collect trajectory experiences, which are then stored in an experience pool. Once the experience pool reaches a preset capacity, the model parameters begin to be updated. First, trajectory data from the experience pool is randomly sampled, and the GTrXL network processes the agent's experiences, encoding the meta-trajectories for each meta-task to obtain state sequences suitable for meta-reinforcement learning algorithms. The GTrXL model encodes the experience sequences into a series of hidden representations, referred to as the outputs of the GTrXL encoder. Each hidden representation contains information about the entire sequence, with positional information encoded as vectors relative to their positions. These hidden representations serve as inputs to the meta-reinforcement learning algorithm. The hidden representations are calculated as follows:

$$c_i = \frac{1}{N} \sum_{n=1}^{N} \text{LayerNorm}(h_{i,n}), \tag{19}$$

where c_i represents the initial state representation for each meta-task, and h_i represents the meta-state sequence.

Next, the meta-reinforcement learning algorithm uses these hidden representations for meta-training. During training, the single-step error is first calculated:

$$\delta_t = r_t + \gamma V_\mu(S_{t+1}) - V_\mu(S_t), \tag{20}$$

where r_t denotes the reward at time step t, $V_\mu(S_{t+1})$ denotes the value function output of the Critic network for state $S_t + 1$ at time step $t+1$, $V_\mu(S_t)$ denotes the value function output of the Critic network for state S_t at time step t, and γ is the discount factor. Then, the network target y_t is calculated, and the Critic network is updated via backpropagation:

$$y_t = \hat{A}_t^{\text{GAE}(\gamma,\lambda)} + V_\mu(S_t), \tag{21}$$

Thus, the loss function can be expressed as:

$$L(\mu) = \frac{\sum_t (y_t - V_\mu(S_t))^2}{\text{batch size}}, \tag{22}$$

Additionally, to accelerate the training process, only one gradient descent step is performed in the inner loop of meta-learning. For each meta-task T_i, meta-reinforcement learning is performed using the initial state representation c_i and the initial policy parameters θ_i, yielding new policy parameters

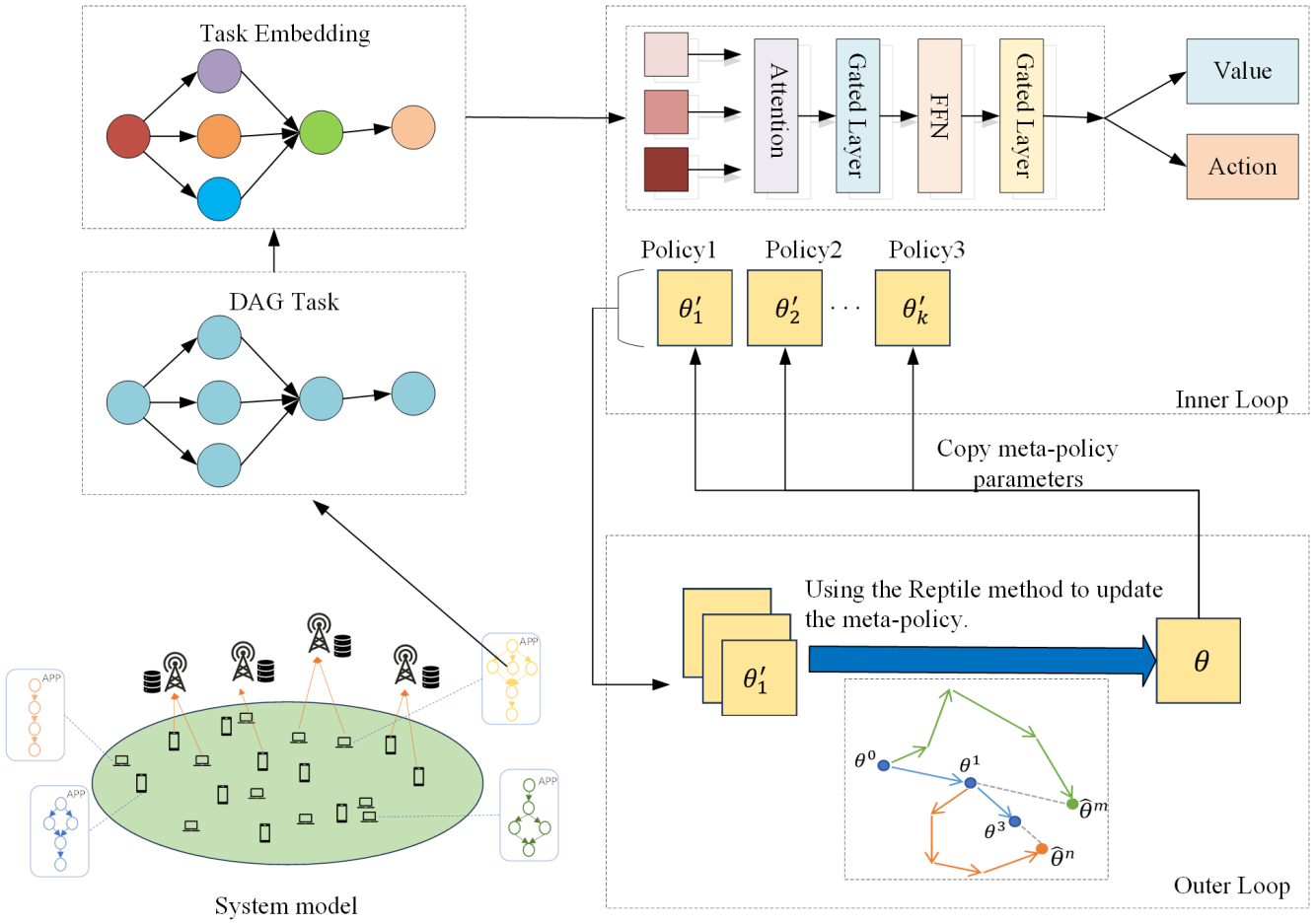

FIGURE 3. Workflow of the GTrXL-PR algorithm.

θ'_i. Therefore, the inner loop update for each task can be represented by the following formulas:

$$\begin{aligned} \theta^{(t)}_{\text{task1}} &\leftarrow \theta_{\text{task1}} - \alpha \frac{\partial L^{\text{clip}}(\theta_m)}{\partial \theta_{\text{task1}}} \\ &\cdots \\ \theta^{(t)}_{\text{task10}} &\leftarrow \theta_{\text{task10}} - \alpha \frac{\partial L^{\text{clip}}(\theta_m)}{\partial \theta_{\text{task10}}}, \end{aligned} \tag{23}$$

where α is the learning rate for the inner loop and $L^{\text{clip}}(\theta_m)$ is the loss function.

After k updates the model parameters in the inner loop, the training process enters the outer loop meta-model training phase. The outer loop obtains the network parameters after the inner loop training and then calculates the update gradients for further updating the meta-model parameters, minimizing the expected loss function across the task distribution, thereby enabling the meta-model to quickly adapt to new tasks. This process can be expressed as follows:

$$\mu_{\text{meta}} = \mu_{\text{meta}} + \epsilon(\tilde{\mu}_\tau - \mu_{\text{meta}}), \tag{24}$$

where μ_{meta} denotes the meta-model parameters before the update, $\tilde{\mu}_\tau$ denotes the parameters learned from the task trajectory τ, and ϵ denotes the model update step size. After the outer loop updates the meta-model parameters, it re-enters the next round of the inner loop, where the parameters of each task model in the next inner loop are trained and updated using the parameters from the meta-model updated in the outer loop.

2) Meta-Testing Phase

The meta-testing phase differs from the meta-training phase. In the meta-testing phase, the "outer loop" does not perform updates. At the beginning of meta-testing, different test tasks from those used in the training phase are selected. Then, the "inner loop" obtains the trained meta-model parameters and uses them as the initial parameters for the inner loop. Finally, the new tasks are subjected to loss calculation and gradient updates according to the "inner loop" update method from the training phase. This process updates the model through backpropagation, enabling the model to adapt to new tasks with only a few updates.

The pseudo-code of the GTrXL-PR algorithm is shown in **Algorithm 2**. In the pseudo-code, θ_j^0 represents the initial policy parameters for the j-th task θ_j represents the current

Algorithm 2 GTrXL-PR

Meta-Training Phase

Randomly initialize meta-policy network parameters θ.
for $i = 1, 2, \ldots, M$ **do**
  Randomly sample N tasks ($\{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_N\}$ from the task distribution $\rho(\mathcal{T})$.
  for $j = 1, 2, ..., N$ **do**
    Initialize the policy for each task $\theta_j^0 \leftarrow \theta, \theta_j \leftarrow \theta$.
    Use the policy $\pi_{\theta_j^0}$ to sample trajectory data $D = \{\tau_1, \tau_2, \ldots, \tau_n\}$ from $\mathcal{T}_j$.
    Encode the trajectories using **Algorithm** 1.
    for $k = 1, 2, ..., L$ **do**
      Use Adam gradient descent to update the policy parameters θ_j for each task using the trajectories:
      $\theta_j \leftarrow \theta_j + \alpha \nabla_{\theta_j} J_{\tau_j}^{\mathrm{PPO}}(\theta_j)$.
    end for
    Update the policy: $\theta_j' \leftarrow \theta_j$.
  end for
  Use Adam to update the meta-policy θ at learning rate β:
  $\theta \leftarrow \theta + \beta \frac{1}{N} \sum_{j=1}^{N} \left[\frac{(\theta_j' - \theta)}{\alpha L} \right]$.
end for

Meta-Testing Phase

for *episode* $= 1, 2, ..., M$ **do**
  Initialize the optimal policy parameters for the task $\theta_0 \leftarrow \theta, \theta_0^0 \leftarrow \theta$.
  Use the policy $\pi_{\theta_0^0}$ to sample trajectory data $D = \{\tau_1, \tau_2, \ldots, \tau_n\}$ from $\mathcal{T}_i$.
  Update the policy θ_i for the specific task using the trajectory and the PPO algorithm with learning rate α.
  for iteration$k = 1, 2, ..., L$ **do**
    Use Adam gradient descent to update the specific task policy parameters θ_0 using the trajectories:
    $\theta_0 \leftarrow \theta_0 + \alpha \nabla_{\theta_0} J_{\tau_0}^{\mathrm{PPO}}(\theta_0)$.
  end for
  Update the policy: $\theta_0' \leftarrow \theta_0$.
end for

policy parameters for the j-th task, $\nabla_{\theta_j} J_{\tau_j}^{\mathrm{PPO}}(\theta_j)$ represents the gradient based on the PPO algorithm, $\theta_j^{'}$ represents the final policy parameters after training for the j-th task, and $\frac{(\theta_j' - \theta)}{\alpha L}$ represents the change in policy parameters for each task.

V. SIMULATION RESULT AND NUMERICAL ANALYSIS

A. SIMULATION SETTINGS

In this experiment, we assume there are four MEC servers and twenty user devices, each with a task that requires computation. Each task is represented by a DAG generated by a DAG generator [40]. For the clock frequency in the offloading scenario, this chapter considers the CPU clock frequency of user devices to be 1×10^9 cycles/s and the edge device clock frequency to be 8×10^9 cycles/s.

In this chapter, we first validate the rapid adaptation and convergence performance of the meta-model under different environmental parameter conditions on the test set. Second, we compare the performance of different meta-learning algorithms. Finally, we compare the performance between different network models.

TABLE 2. Experimental Parameter Settings.

Parameter Definitions	Value
Number of Meta-Training Tasks (Count)	100
Number of Meta-Testing Tasks (Count)	100
Communication-to-Computation Ratio(CCR)	0.3∼0.5
DAG Fat(Fat)	{0.1, 0.3, 0.4, 0.5, 0.7}
DAG Density(Density)	{0.4, 0.5, 0.6, 0.7, 0.8}
The inner loop learning rate	0.1
The outer loop learning rate	0.01

B. ANALYSIS OF RAPID ADAPTATION CAPABILITIES IN DIFFERENT SCENARIOS

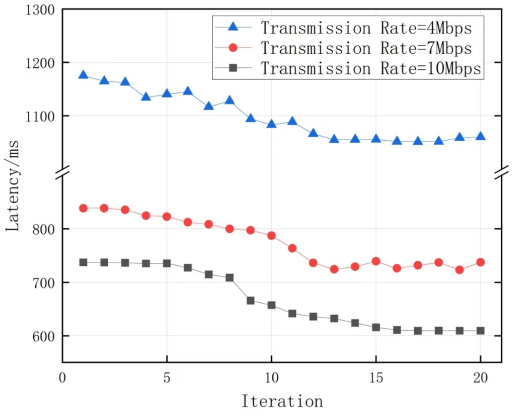

FIGURE 4. Rapid Adaptation Performance at Different Transmission Rates.

First, we consider a scenario with multiple applications and multiple edge devices, where each application can be subdivided into 20 fine-grained tasks. The test set contains 100 DAGs, representing 100 applications. The trained meta-model also exhibits good convergence performance for different transmission rates, requiring only a few iterations to quickly converge and adapt to new environments. As shown in Fig. 4, with uplink and downlink rates of 4, 7, and 10 Mbps, the task offloading latency decreases with the increase in transmission rate, and the system quickly reaches a converged state. This indicates that the trained meta-model can effectively handle different transmission rates in real environments. The above experiments only demonstrate that the proposed algorithm can adapt well to new environments and reach convergence. Next, by executing different algorithms, we compare the proposed algorithm to evaluate its optimization effectiveness on decision latency and rapid adaptation. To further validate the rapid adaptation capabilities of the meta-model, we conduct experiments in environments with

different numbers of tasks. As shown in Fig. 5, we compare Greedy, GASTO [41], MRLCO [38], T-PPO, and the GTrXL-PR proposed in this paper. The algorithm descriptions are as follows:

- Greedy: the subtasks of DAG tasks are offloaded based on the estimated minimum execution time.
- MRLCO: a meta-reinforcement learning algorithm using MAML.
- GASTO: a meta-reinforcement learning algorithm that extracts the internal spatial features of the DAG using graph neural networks and performs offloading decisions through a seq2seq network.
- T-PPO: a transfer-reinforcement learning algorithm using PPO.

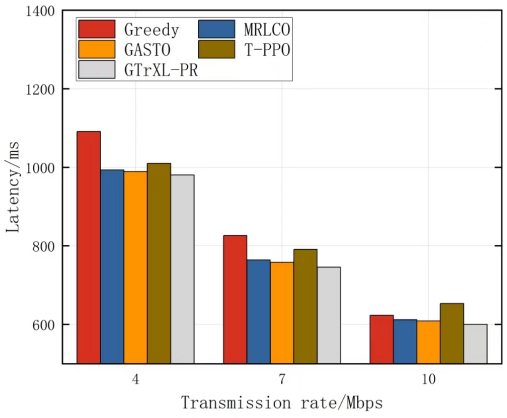

FIGURE 5. Comparison of Algorithms under Different Transmission Rates

To validate the advantages of the proposed algorithm at different transmission rates, Fig. 5 shows that the latency of several algorithms decreases with the increase in transmission rate. Overall, the latency of GASTO and MRLCO is slightly higher than that of the proposed algorithm, but both are superior to the Greedy algorithm and the T-PPO algorithm. Due to the low computational capability of local execution, the resulting latency is relatively high. Although fully offloading the tasks to the edge with higher computational capability can reduce computational latency, it incurs significant transmission latency. Therefore, the optimal approach is partial offloading, which fully utilizes both local and edge computing resources. As shown in Fig. 5, this chapter uses the Greedy algorithm, MRLCO, GASTO, and the T-PPO algorithm as comparison algorithms for the proposed algorithm. The Greedy algorithm selects the current optimal solution rather than the global optimal solution, which may lead to local optima. The structure used in the proposed GTrXL-PR algorithm incorporates self-attention mechanisms and parallel processing, which facilitates smoother gradient propagation and faster convergence. Additionally, the proposed GTrXL-PR algorithm employs the Reptile algorithm, resulting in a total offloading latency similar to other algorithms but with a shorter training time. This is because Reptile does not require the computation of higher-order derivatives, thus reducing training time. Although the Reptile algorithm does not perform gradient descent in the outer loop, it normalizes the gradients from the inner loop and updates the initial parameters accordingly, without discarding parameters that would degrade performance.

C. COMPARISON OF DIFFERENT TOPOLOGIES

First, to validate the generalization capability of the proposed algorithm under different topological properties constructed within different DAGs, we selected three sets of density and width combinations {0.1, 0.5}, {0.4, 0.6}, and {0.7, 0.4} for testing, while keeping other parameters constant. These values are based on the densities and widths listed in Table 2 during the training process. By setting different densities and widths, DAGs with various topological structures can be obtained. The results are shown in Fig. 6.

As can be seen from the figure, when the DAG width and density change, the proposed algorithm outperforms the comparison algorithms given the same number of iterations. This is because the GTrXL structure adopted in this paper can capture the hidden space of states, thus better adapting to different situations. Additionally, to validate generalization capability, we conducted experiments with a density and width combination 0.4, 0.9 that was not present in the training data.

Observing Fig. 7, the three algorithms maintained similar performance from the beginning. Both MRLCO and GASTO, like the proposed algorithm, adopt the concept of meta-learning and have strong generalization capabilities. Task processing delays monotonically decrease with training iterations. However, it can be further observed that the proposed algorithm converges significantly faster, achieving better latency performance.

D. COMPARISON OF DIFFERENT NETWORK ARCHITECTURES

Different network architectures can affect the execution time of the learned policy in various ways. This chapter uses the GTrXL network architecture, which can leverage the self-attention mechanism to compute the importance of each experience in the sequence for the task, thereby generating corresponding state vectors. In the specific scenario of meta-learning, each task has only a few samples, making Long Short-Term Memory (LSTM) prone to overfitting, which results in poor generalization performance. GTrXL, on the other hand, can better capture dependencies in sequence data, which can lead to better generalization performance in some tasks, reducing the number of meta-learning iterations and overall training time. Furthermore, since the attention mechanism in GTrXL can better capture key information in sequences, it can help the model learn commonalities among tasks more effectively during the meta-training phase, thus enhancing the effectiveness of meta-learning. GRU, due to

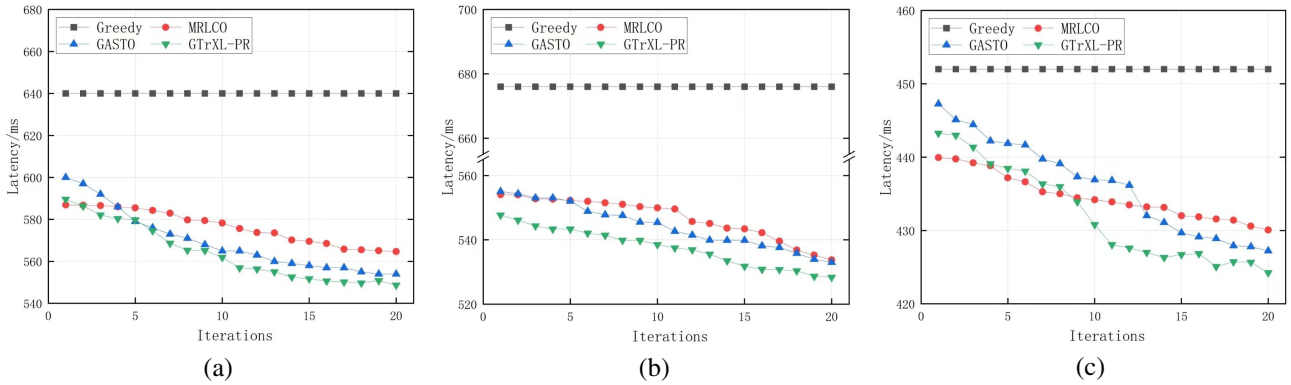

FIGURE 6. Comparison of Different Topologies: (a) density=0.7, width=0,4; (b) density=0.4, width=0,6; (c) density=0.1, width=0.5

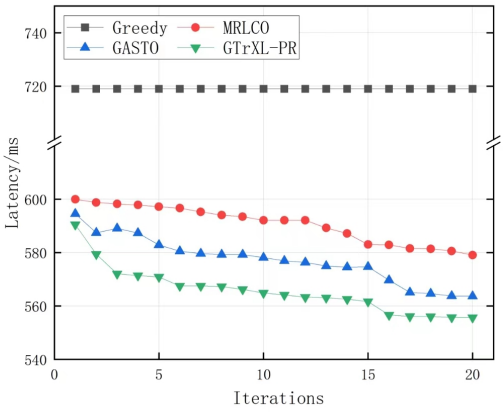

FIGURE 7. Comparison of Generalization across Different Topologies

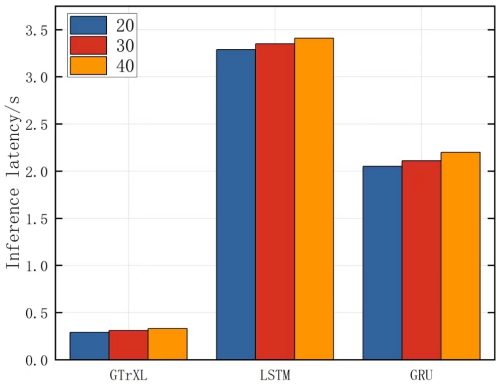

FIGURE 8. Comparison of Generalization across Varying Task Numbers

its relatively simple structure with fewer parameters, can be faster than LSTM in some cases. GTrXL can effectively capture global relationships in experience sequences and avoid the computational bottleneck caused by sequential iterations in LSTM, thereby speeding up inference. As shown in Fig. 8, the inference time is significantly reduced compared to LSTM and GRU network structures under different numbers of tasks.

VI. CONCLUSIONS

This paper proposes a meta-reinforcement learning-based task offloading decision framework, GTrXL-PR, to address the deficiencies of existing methods in MEC environments regarding sample efficiency, model adaptability, training time, and stability. By combining the powerful feature-capturing capabilities of Transformer-XL with the efficient training mechanisms of the reptile and PPO algorithms, we achieve effective task offloading decisions in complex dynamic environments.

The results indicate that the GTrXL-PR framework demonstrates significant performance improvements during the task offloading process, particularly in handling task variations in dynamic environments. Specifically, the framework is capable of quickly adapting to new tasks, significantly enhancing decision-making efficiency by effectively capturing the complex features and relationships among tasks. This finding suggests that the introduction of meta-reinforcement learning enables the model to accumulate experience from historical tasks, allowing for rapid strategy adjustments when facing new and evolving tasks. Through this innovative approach, we not only address the limitations of existing technologies but also provide a reliable solution for practical applications in MEC.

In extensive experimental validations, the GTrXL-PR framework consistently demonstrated superior performance across various typical MEC task offloading scenarios. It significantly outperformed existing methods in terms of adaptability, inference time, and overall performance. The experimental results indicate that our solution not only achieves substantial performance improvements but also offers high reliability and scalability.

However, this research does have some limitations that should be acknowledged. First, while the GTrXL-PR framework shows promising performance in typical MEC scenar-

ios, its effectiveness in highly heterogeneous environments with extreme variability in task types and resource availability remains to be further explored. Moreover, while we have focused on optimizing task-offloading decisions, the integration of other critical factors such as energy consumption, and user satisfaction into the decision-making process could further enhance the robustness of our framework.

In summary, this research provides an efficient and reliable solution for task offloading decisions in MEC environments. By integrating meta-reinforcement learning and deep learning technologies, our GTrXL-PR framework effectively addresses the key issues of existing methods, promoting the development and adoption of MEC technology in practical applications. In the future, we will further optimize the model structure and algorithms by incorporating energy consumption as a factor, considering the dual objectives of latency and energy efficiency to tackle more complex task offloading scenarios, and explore its application potential in other computing paradigms.

• • •

REFERENCES

[1] C. Feng, P. Han, X. Zhang, B. Yang, Y. Liu, and L. Guo, "Computation offloading in mobile edge computing networks: A survey," *Journal of Network and Computer Applications*, vol. 202, p. 103366, 2022.

[2] L. Dong, F. Jiang, M. Wang, Y. Peng, and X. Li, "Deep progressive reinforcement learning-based flexible resource scheduling framework for irs and uav-assisted mec system," *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[3] F. Jiang, Y. Peng, L. Dong, K. Wang, K. Yang, C. Pan, D. Niyato, and O. A. Dobre, "Large language model enhanced multi-agent systems for 6g communications," *IEEE Wireless Communications*, 2024.

[4] W. Zhou, L. Xing, J. Xia, L. Fan, and A. Nallanathan, "Dynamic computation offloading for mimo mobile edge computing systems with energy harvesting," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 5, pp. 5172–5177, 2021.

[5] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "Ai driven heterogeneous mec system with uav assistance for dynamic environment: Challenges and solutions," *IEEE Network*, vol. 35, no. 1, pp. 400–408, 2020.

[6] M. Y. Akhlaqi and Z. B. M. Hanapi, "Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions," *Journal of Network and Computer Applications*, vol. 212, p. 103568, 2023.

[7] L. Dong, Z. Liu, F. Jiang, and K. Wang, "Joint optimization of deployment and trajectory in uav and irs-assisted iot data collection system," *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21 583–21 593, 2022.

[8] F. Jiang, K. Wang, L. Dong, C. Pan, and K. Yang, "Stacked autoencoder-based deep reinforcement learning for online resource scheduling in large-scale mec networks," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9278–9290, 2020.

[9] H. Qiu, K. Zhu, N. C. Luong, C. Yi, D. Niyato, and D. I. Kim, "Applications of auction and mechanism design in edge computing: A survey," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 2, pp. 1034–1058, 2022.

[10] F. Jiang, L. Dong, K. Wang, K. Yang, and C. Pan, "Distributed resource scheduling for large-scale mec systems: A multiagent ensemble deep reinforcement learning with imitation acceleration," *IEEE Internet of Things Journal*, vol. 9, no. 9, pp. 6597–6610, 2021.

[11] L. Dong, Y. Peng, F. Jiang, K. Wang, and K. Yang, "Explainable semantic federated learning enabled industrial edge network for fire surveillance," *IEEE Transactions on Industrial Informatics*, 2024.

[12] W. Zhan, C. Luo, G. Min, C. Wang, Q. Zhu, and H. Duan, "Mobility-aware multi-user offloading optimization for mobile edge computing," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 3, pp. 3341–3356, 2020.

[13] L. Dong, F. Jiang, Y. Peng, K. Wang, K. Yang, C. Pan, and R. Schober, "Lambo: Large ai model empowered edge intelligence," *IEEE Communications Magazine*, 2024.

[14] R. Vilalta and Y. Drissi, "A perspective view and survey of meta-learning," *Artificial intelligence review*, vol. 18, pp. 77–95, 2002.

[15] M. Silvetti, S. Lasaponara, N. Daddaoua, M. Horan, and J. Gottlieb, "A reinforcement meta-learning framework of executive function and information demand," *Neural Networks*, vol. 157, pp. 103–113, 2023.

[16] B. Rong, "Energy efficient computation offloading in mobile edge computing," *IEEE Wireless Communications*, vol. 30, no. 2, pp. 8–8, 2023.

[17] M. Cavus, Y. F. Ugurluoglu, H. Ayan, A. Allahham, K. Adhikari, and D. Giaouris, "Switched auto-regressive neural control (s-anc) for energy management of hybrid microgrids," *Applied Sciences*, vol. 13, no. 21, p. 11744, 2023.

[18] M. Cavus, A. Allahham, K. Adhikari, and D. Giaouris, "A hybrid method based on logic predictive controller for flexible hybrid microgrid with plug-and-play capabilities," *Applied Energy*, vol. 359, p. 122752, 2024.

[19] L. Zhang, Y. Sun, Z. Chen, and S. Roy, "Communications-caching-computing resource allocation for bidirectional data computation in mobile edge networks," *IEEE Transactions on Communications*, vol. 69, no. 3, pp. 1496–1509, 2020.

[20] Q.-V. Pham, F. Fang, V. N. Ha, M. J. Piran, M. Le, L. B. Le, W.-J. Hwang, and Z. Ding, "A survey of multi-access edge computing in 5g and beyond: Fundamentals, technology integration, and state-of-the-art," *IEEE access*, vol. 8, pp. 116 974–117 017, 2020.

[21] L. Huang, S. Bi, and Y.-J. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 19, no. 11, pp. 2581–2593, 2019.

[22] C. Sun, X. Wu, X. Li, Q. Fan, J. Wen, and V. C. Leung, "Cooperative computation offloading for multi-access edge computing in 6g mobile networks via soft actor critic," *IEEE Transactions on Network Science and Engineering*, 2021.

[23] F. Jiang, L. Dong, Y. Peng, K. Wang, K. Yang, C. Pan, and X. You, "Large ai model empowered multimodal semantic communications," *IEEE Communications Magazine*, 2024.

[24] F. Jiang, Y. Peng, L. Dong, K. Wang, K. Yang, C. Pan, and X. You, "Large ai model-based semantic communications," *IEEE Wireless Communications*, vol. 31, no. 3, pp. 68–75, 2024.

[25] Y. Chen, Z. Liu, Y. Zhang, Y. Wu, X. Chen, and L. Zhao, "Deep reinforcement learning-based dynamic resource management for mobile edge computing in industrial internet of things," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 7, pp. 4925–4934, 2020.

[26] X. He, H. Lu, M. Du, Y. Mao, and K. Wang, "Qoe-based task offloading with deep reinforcement learning in edge-enabled internet of vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 4, pp. 2252–2261, 2020.

[27] G. Qu, H. Wu, R. Li, and P. Jiao, "Dmro: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 3448–3459, 2021.

[28] S. Chen, L. Rui, Z. Gao, W. Li, and X. Qiu, "Cache-assisted collaborative task offloading and resource allocation strategy: A metareinforcement learning approach," *IEEE Internet of Things Journal*, vol. 9, no. 20, pp. 19 823–19 842, 2022.

[29] P. Dai, Y. Huang, K. Hu, X. Wu, H. Xing, and Z. Yu, "Meta reinforcement learning for multi-task offloading in vehicular edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 3, pp. 2123–2138, 2023.

[30] M. A. Hossain, W. Liu, and N. Ansari, "Computation-efficient offloading and power control for mec in iot networks by meta reinforcement learning," *IEEE Internet of Things Journal*, 2024.

[31] Y. Yang, C. Shao, J. Zuo, and C. Shi, "Energy efficient algorithm for multi-user adaptive edge computing offloading in vehicular networks based on meta reinforcement learning," in *2024 7th World Conference on Computing and Communication Technologies (WCCCT)*. IEEE, 2024, pp. 250–254.

[32] E. Parisotto, F. Song, J. Rae, R. Pascanu, C. Gulcehre, S. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, S. Noury *et al.*, "Stabilizing transformers for reinforcement learning," in *International conference on machine learning*. PMLR, 2020, pp. 7487–7498.

[33] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-xl: Attentive language models beyond a fixed-length context," *arXiv preprint arXiv:1901.02860*, 2019.

[34] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in uav-assisted mobile edge computing," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 6949–6960, 2022.

[35] Y. Yang, R. Yan, and Y. Gu, "Ltransformer: A transformer-based framework for task offloading in vehicular edge computing," *Applied Sciences*, vol. 13, no. 18, p. 10232, 2023.

[36] N. Gholipour, M. D. de Assuncao, P. Agarwal, J. Gascon-Samson, and R. Buyya, "Tpto: A transformer-ppo based task offloading solution for edge computing environments," in *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2023, pp. 1115–1122.

[37] C.-L. Wu, T.-C. Chiu, C.-Y. Wang, and A.-C. Pang, "Mobility-aware deep reinforcement learning with seq2seq mobility prediction for offloading and allocation in edge computing," *IEEE Transactions on Mobile Computing*, 2023.

[38] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 242–253, 2020.

[39] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: reinforcement learning via sequence modeling, 2 june 2021," *URL http://arxiv.org/abs/2106.01345*.

[40] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE transactions on parallel and distributed systems*, vol. 25, no. 3, pp. 682–694, 2013.

[41] Y. Li, J. Li, Z. Lv, H. Li, Y. Wang, and Z. Xu, "Gasto: A fast adaptive graph learning framework for edge computing empowered task offloading," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 932–944, 2023.

YINGTAO LI received the M.E. degree in Computer Science and Technology from Hunan Normal University, China, in 2024. Her research interests include machine learning and mobile edge computing.

CHUANGUO TANG received the B.E. degree in Computer Science and Technology from Hunan University of Science and Technology, Xiangtan, China, in 2022. He is currently pursuing the M.E. degree in Hunan Normal University, Changsha. His research interests include machine learning and mobile edge computing.

BOHAO HOU Bohao Hou currently pursuing the M.E. degree in information and communication engineering with Hunan University of Technology and Business, Changsha. His research interests include machine learning and mobile edge computing

FEIBO JIANG received his B.S. and M.S. degrees in School of Physics and Electronics from Hunan Normal University, China, in 2004 and 2007, respectively. He received his Ph.D. degree in School of Geosciences and Info-physics from the Central South University, China, in 2014. He is currently an associate professor at the Hunan Provincial Key Laboratory of Intelligent Computing and Language Information Processing, Hunan Normal University, China. His research interests include artificial intelligence, fuzzy computation, Internet of Things, and mobile edge computing.

YUHANG FU received the B.S. M.S. degree in School of Physics and Electronics from Hunan Normal University, China, in 2004. His research interests include machine learning and mobile edge computing.